LLM-Driven Quadruped Robot Inspection System with Human-in-the-Loop Control via Model Context Protocol

Muhamad Rizano Alamsyah
Department of Computer Engineering
Institut Teknologi Sepuluh Nopember
Surabaya, Indonesia
ershanoano@email.com

Muhtadin, S.T., M.T.
Department of Computer Engineering
Institut Teknologi Sepuluh Nopember
Surabaya, Indonesia
muhtadin@its.ac.id

Ir. Hany Boedinugroho, M.T.
Department of Computer Engineering
Institut Teknologi Sepuluh Nopember
Surabaya, Indonesia
hanny@its.ac.id

***Abstract*—Industrial inspection using quadruped robots presents a challenge in balancing operational autonomy with the adaptability needed for dynamic environments. Existing approaches are either fully teleoperated—imposing high cognitive load on operators—or fully autonomous with rigid, pre-programmed plans that cannot respond to semantic human commands or unexpected field conditions. This paper presents a distributed inspection system integrating a Large Multimodal Model (LMM), specifically Google Gemini, as an agentic orchestrator for a quadruped robot (DEEPRobotics Jueying Lite3 Pro). The architecture employs the Model Context Protocol (MCP) to bridge the LLM agent with a ROS-based navigation stack, enabling semantic instruction processing, autonomous navigation, and SOP-based visual inspection. A stepwise Human-in-the-Loop (HITL) mechanism—facilitated through webhook callbacks—ensures operator validation at each mission stage while preserving system adaptability. The system further applies Retrieval-Augmented Generation (RAG) to ground LLM decisions in structured operational data (waypoints and SOP documents). Experimental results demonstrate the system's ability to autonomously plan and execute multi-waypoint inspection missions from natural language commands with accurate SOP compliance checking.**

***Index Terms*—quadruped robot, large language model, agentic AI, human-in-the-loop, model context protocol, ROS, autonomous inspection**

I. Introduction

The rapid growth of autonomous mobile robotics—projected to reach USD 9.56 billion by 2030 [1]—reflects increasing demand for automation in high-risk industrial environments. Quadruped robots have emerged as particularly capable platforms for such environments due to their superior terrain adaptability compared to wheeled alternatives [2]. However, despite this mobility advantage, the deployment of quadruped robots for industrial inspection remains operationally challenging.

Current inspection paradigms fall into two extremes. *Teleoperation* reduces operator risk but imposes continuous cognitive demand: operators must simultaneously interpret raw telemetry and visual feeds while issuing motor commands [3]. *Fully autonomous* approaches address this through deterministic navigation to pre-programmed waypoints, as demonstrated by Darmawan [4], but remain inflexible—any change in inspection scope requires code-level reprogramming.

Initial attempts at integrating Large Language Models (LLMs) for robot control, such as that proposed by Mahendra [5], offer natural language flexibility but generate and execute complete action plans non-interactively. This "batch-execution" approach provides no mechanism for operator intervention mid-mission, which is critical in dynamic industrial settings where conditions may change unexpectedly [6].

This paper addresses these limitations through three contributions:

1) A distributed agentic architecture connecting a multimodal LLM to a ROS-based robot controller via the Model Context Protocol (MCP), enabling semantic command interpretation and tool-based execution.
2) A stepwise Human-in-the-Loop (HITL) mechanism using webhook callbacks that pauses execution after each robot action, awaiting operator validation before proceeding.
3) A Retrieval-Augmented Generation (RAG) pipeline that grounds LLM decisions in structured inspection data (waypoint coordinates and SOP documents) retrieved on demand.

II. Related Work

This research builds upon and addresses the limitations of two primary antecedent systems in quadruped robotics for industrial inspection.

A. Deterministic Autonomous Inspection

Darmawan [4] developed an autonomous monitoring system using a quadruped robot to estimate the position of overheating components in substations. The system successfully integrated thermal cameras with YOLOv8s object detection and utilized a robust ROS-based navigation stack (FastLIO2 for SLAM and Dijkstra for global planning). While highly reliable in execution, the system's navigation was fundamentally deterministic. The robot could only navigate to explicitly hard-coded coordinates, offering no flexibility to adapt to dynamic

scenarios or semantic human instructions without code-level reprogramming by technical personnel.

B. LLM-Based Movement Planning

To introduce semantic flexibility, Mahendra [5] implemented a natural language interaction system using Google Gemini to autonomously generate movement plans for the Jueying Lite3 quadruped robot. The LLM parsed user commands into a complete JSON sequence of navigation actions. However, the system utilized a "batch-execution" paradigm: the entire multi-waypoint plan was generated upfront and executed sequentially without interruption. This architecture lacked a step-by-step validation mechanism, stripping operators of the ability to intervene, modify the plan mid-mission, or verify intermediate inspection results before the robot proceeded to the next target.

C. Proposed Contribution

To bridge the gap between flexible language understanding and safe, interactive execution, our work introduces an agentic architecture via the Model Context Protocol (MCP). Unlike [5], our system does not generate a static sequence of actions. Instead, the LLM agent iteratively invokes robotic tools and evaluates their outcomes. By architecturally enforcing a stepwise Human-in-the-Loop (HITL) mechanism, operators can validate or alter the inspection trajectory after every waypoint, while a Retrieval-Augmented Generation (RAG) pipeline ensures the LLM's decisions are grounded in structured operational data.

III. System Architecture

A. Overview

The system adopts a distributed architecture comprising four loosely coupled components, as illustrated in Fig. 1. Communication between components follows explicit protocols to ensure modularity and replaceability.

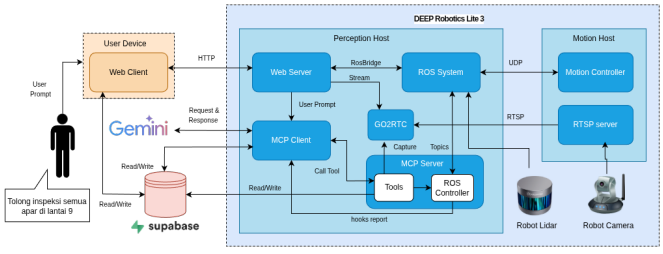

Fig. 1. System architecture. The LLM agent (MCP Client) bridges the operator interface and the robot-side MCP Server via the Model Context Protocol. Cloud services (Gemini API and Supabase) are accessed through the internet.

- **Web Dashboard (robodog-web):** A Next.js application serving as the operator control station. It communicates with the backend via REST API for chat and data management. Real-time telemetry (robot pose, occupancy map) is streamed directly from the ROS ecosystem via *rosbridge* WebSocket. Camera video is delivered over WebRTC.
- **LLM Agent / MCP Client (robodog-client):** A FastAPI service that acts as the agentic middleware. It holds the system prompt defining the inspection workflow, manages conversation history in Supabase, and implements the agentic loop: forward prompts to Gemini, intercept *function_call* responses, dispatch tool calls to the MCP Server, and inject results back into the LLM context.
- **MCP Server (robodog-server):** A FastMCP HTTP server running on the robot's onboard AI computer (NVIDIA Jetson Xavier NX). It exposes six tools (Table II) and handles all robot-side execution including ROS interaction and database retrieval.
- **Cloud Services:** Google Gemini API provides LLM inference. Supabase provides PostgreSQL database storage (sessions, messages, locations, waypoints, objects) and file storage (SOP documents, captured images, maps).

B. Communication Protocols

Table I summarizes the inter-component communication.

TABLE I
COMMUNICATION PROTOCOLS BETWEEN SYSTEM COMPONENTS

Link	Protocol	Purpose
Web ↔ Backend	REST / HTTP	Chat messages, CRUD
Web ↔ ROS	WebSocket (rosbridge)	Telemetry, maps
Web ↔ Robot camera	WebRTC	Live video
Backend ↔ MCP Server	MCP over HTTP/SSE	Tool invocation
MCP Server ↔ ROS	rospy (internal)	cmd_vel, move_base
Robot → Backend	HTTP POST (webhook)	Action completion report
Backend ↔ Gemini	Google GenAI SDK	LLM inference
All ↔ Supabase	REST API / SDK	Database & storage

C. Hardware Platform

The robot platform is the DEEPRobotics Jueying Lite3 Pro [14], a 12-DOF quadruped (mass 12.7 kg, 610×370×445 mm standing), augmented with a Leishen C16 16-channel 3D LiDAR [15]. The onboard AI computer is an NVIDIA Jetson Xavier NX, which runs all software components (ROS stack, MCP Server, MCP Client, and web application) as a single-host deployment managed by *systemd* services and Docker Compose. Remote access is provided through Tailscale VPN, eliminating the need for a shared local network.

IV. System Design

A. MCP Tool Definitions

The MCP Server exposes six tools that collectively span the full capability surface of the robot, as detailed in Table II. Tool schemas are converted from MCP format to Gemini *FunctionDeclaration* format at runtime.

B. Agentic Loop and HITL Mechanism

The inspection workflow is orchestrated by an agentic loop inside the MCP Client, as shown in Fig. 2. The loop operates as follows:

TABLE II
MCP TOOLS EXPOSED BY THE ROBOT SERVER

Tool	Description
`move`	Manual open-loop velocity control (speed, duration)
`navigate_to_waypoint`	Autonomous navigation via *move_base* Action (x, y, θ)
`get_object_waypoints`	Hierarchical DB query: location → object → waypoints
`list_sop_files`	List SOP files in Supabase Storage
`get_sop_file`	Download and return SOP document bytes
`capture_and_upload_image`	Capture camera frame via RTSP, upload to Supabase, return URL

1) The operator issues a natural language command (e.g., *"Inspect all fire extinguishers in Zone A"*).
2) The backend loads conversation history from Supabase and forwards the full context to Gemini API.
3) Gemini responds with a *function_call*. The backend dispatches the call to the MCP Server.
4) If the tool involves data retrieval (e.g., `get_object_waypoints`, `get_sop_file`), the MCP Server queries Supabase. File bytes (SOP PDFs, images) are injected directly into the Gemini context as `Part.from_bytes` for multimodal analysis.
5) If the tool involves robot action (e.g., `navigate_to_waypoint`), execution is asynchronous: the MCP Server initiates the *move_base* goal and immediately returns. Upon goal completion (or failure), the ROS controller fires an HTTP POST webhook to the MCP Client.
6) The webhook triggers a new Gemini invocation with the action result. Gemini presents findings to the operator and **pauses** for explicit confirmation before proceeding to the next waypoint (HITL gate).
7) The loop repeats until the full inspection plan is executed or the operator aborts.

C. RAG-Based Operational Grounding

Rather than encoding inspection knowledge in the system prompt, the architecture uses tool-time retrieval. When Gemini identifies an inspection target, it calls `get_object_waypoints` to fetch spatial coordinates from the database and `get_sop_file` to retrieve the relevant SOP document. This approach ensures that:

- The LLM always operates on the latest operational data without redeployment.
- Inspection procedures can be updated by operators (uploading new SOP files) without modifying any code.
- Hallucination risk is reduced as spatial data is retrieved from a ground-truth database rather than generated.

D. Navigation Stack

The ROS Navigation Stack is configured with a Dijkstra-based global planner and a Timed Elastic Band (TEB) local planner [11]. Localization is performed by HDL Localization using NDT scan matching [12] and Fast-GICP [13] against a 3D point cloud map built by Faster-LIO (LiDAR-Inertial Odometry). The 3D map is converted to a 2D occupancy grid via `pcd_2_gridmap` for use by the Navigation Stack.

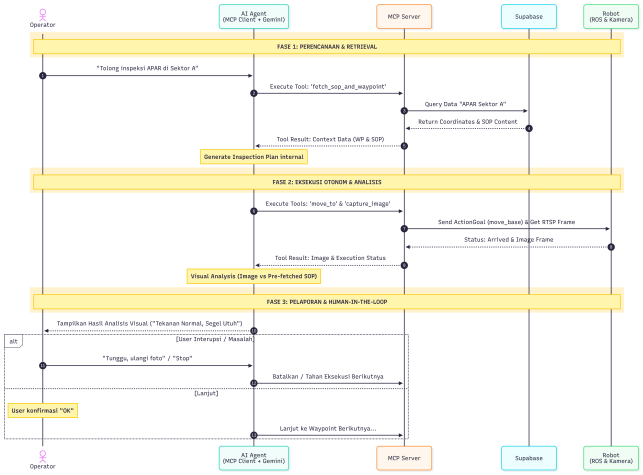

Fig. 2. Sequence diagram of the inspection workflow showing the three-phase agentic loop: (1) Retrieval & Planning, (2) Autonomous Execution & Analysis, (3) HITL Reporting.

V. IMPLEMENTATION

A. Deployment

All software components are deployed on the robot's Jetson Xavier NX. Four *systemd* services manage the core stack: LiDAR driver, navigation (*move_base* + localization), MCP Server (port 8001), and MCP Client/Backend (port 8082). The web dashboard runs in a Docker Compose container (port 3000). Tailscale VPN provides a stable overlay network; the Supabase instance runs on the developer's laptop, mimicking a cloud deployment without public infrastructure requirements.

B. System Prompt Engineering

The system prompt instructs Gemini to follow a strict inspection protocol:

1) Retrieve the relevant SOP document before any navigation.
2) Query waypoints for the target object.
3) Present a step-by-step inspection plan and *wait for operator approval*.
4) Execute waypoints sequentially, one at a time.
5) After each capture, analyze the image against the SOP and report findings.
6) Ask the operator whether to continue to the next waypoint.

This prompt-level enforcement of the HITL protocol is complemented by the architectural webhook mechanism.

C. Web Dashboard

The operator dashboard (Fig. 3) is a single-page Next.js application with eight functional pages. The primary *Inspection Hub* page features a split-screen layout: the left panel shows a live map (rendered from ROS topics via roslibjs) with the robot's real-time pose, and the right panel provides the LLM chat interface. Supporting pages provide management interfaces for spatial locations, inspection objects, SOP documents, captured image galleries, and session history.

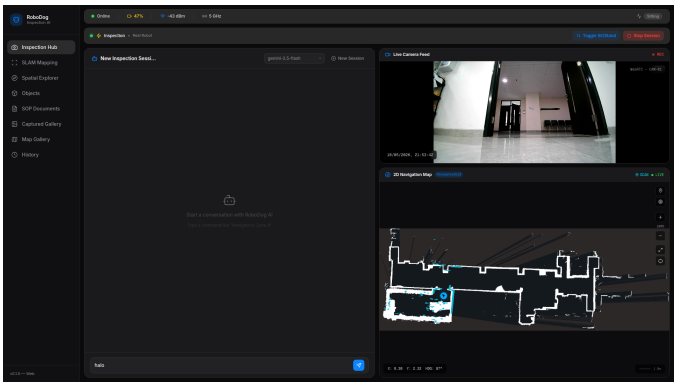

Fig. 3. Web dashboard Inspection Hub: split-screen layout with real-time navigation map (left) and LLM chat panel (right).

VI. Evaluation

A. Experimental Setup

Experiments were conducted in an indoor laboratory environment replicating an industrial inspection scenario containing multiple objects (fire extinguishers, electrical panels, and trash cans). The primary orchestrator for the system was Google Gemini 3.1 Pro via cloud API, which demonstrated the highest reasoning capability. Gemini 2.5 Flash and a locally hosted Qwen 3.5 27B model were also tested as comparative and future-proofing alternatives to ensure the MCP architecture could generalize to lighter or fully localized models. The evaluation was divided into modular capability tests and a full end-to-end (E2E) integration test.

B. Tool Calling and Intent Recognition

The foundation of the MCP architecture relies on the model's ability to map natural language to specific JSON schema tool calls. The models were tested across three difficulty levels: explicit commands (Level 1), implicit context (Level 2), and abstract spatial reasoning (Level 3). Gemini 3.1 Pro achieved a 100% success rate across all levels, consistently invoking the correct tools (e.g., `navigate_to_waypoint`) with valid parameters. Gemini 2.5 Flash and Qwen 3.5 performed flawlessly on Level 1 and 2 but struggled on Level 3. Flash frequently bypassed tool invocation in favor of generating direct conversational responses, while Qwen 3.5 exhibited hallucination by inventing non-existent waypoints instead of querying the database.

C. Spatial Reasoning and Route Planning

A critical requirement for autonomous inspection is calculating the shortest path among multiple targets to solve the Traveling Salesperson Problem (TSP) using zero-shot reasoning. Gemini 3.1 Pro successfully deduced optimal routes in 100% of the complex multi-waypoint scenarios, leveraging spatial coordinate data retrieved from the database. Conversely, Gemini 2.5 Flash suffered a severe performance degradation on complex tasks, generating excessive *Chain-of-Thought* scratchpads (spiking latency to $\sim$42 seconds) and adopting a greedy approach that resulted in suboptimal routes. Qwen 3.5 struggled significantly with multi-waypoint logic, frequently instructing the robot to backtrack unnecessarily.

D. Multimodal Anomaly Detection

Visual inspection relied on the models' ability to cross-reference real-time camera captures with retrieved SOP documents. Gemini 3.1 Pro correctly identified 100% of the anomalies (e.g., missing fire extinguisher hoses, scattered trash) and successfully ignored visual distractors (e.g., a green bag near a green trash can). Qwen 3.5, running locally, experienced a significant latency overhead (median $\sim$35 seconds) due to local vision encoding and failed on high-resolution details, though it successfully identified major anomalies.

E. End-to-End System Performance

The full system integration was tested across 27 distinct scenarios spanning *Happy Paths*, *Anomaly Paths*, and *Human-in-the-Loop (HITL) Paths*. The HITL mechanism proved highly effective in dynamic scenarios. For instance, when an operator interrupted a mission to inject a new inspection target, the webhook-paused agentic loop successfully accepted the new prompt, queried the new coordinates, and seamlessly resumed navigation without requiring a system restart. Gemini 3.1 Pro demonstrated near-perfect execution across all E2E scenarios, proving that the combination of MCP tool orchestration and webhook-based HITL can safely govern quadruped robot operations in unpredictable environments.

VII. Conclusion

This paper presented an LLM-driven quadruped robot inspection system that addresses the limitations of both hard-coded deterministic navigation and non-interactive LLM planning. By combining the Model Context Protocol for structured tool invocation, webhook-based asynchronous reporting, and a stepwise Human-in-the-Loop validation mechanism, the system enables flexible and safe inspection missions from natural language commands. Experimental results demonstrated that advanced cloud models like Gemini 3.1 Pro can successfully govern complex spatial routing (TSP) and visual anomaly detection, while the architecture itself remains robust enough to integrate local open-source models (e.g., Qwen 3.5) for future-proofing against network outages. The RAG-based grounding approach ensures that LLM decisions are anchored to authoritative operational data, reducing hallucination risks. Future

work will focus on extending the system to outdoor environments, integrating thermal imaging for anomaly detection, and optimizing local vision-language models to reduce inference latency on edge computing devices.

REFERENCES

[1] Grand View Research, "Autonomous Mobile Robot Market Size, Share & Trends Analysis Report," 2024. [Online]. Available: https://www.grandviewresearch.com

[2] J. Fan et al., "A review of quadruped robots and environment perception," *Sensors*, vol. 24, no. 4, 2024.

[3] D. Rea and C. Anagnostopoulos, "Is teleoperation still relevant for human-robot interaction?" *ACM/IEEE HRI*, 2022.

[4] Darmawan, "Pengembangan Robot Quadruped untuk Estimasi Posisi Komponen Overheat Pada Gardu Induk Berbasis Kamera Termal," Tugas Akhir, Institut Teknologi Sepuluh Nopember, 2025.

[5] V. G. P. A. B. Mahendra, "Interaksi Robot Manusia Berbasis Large Language Model untuk Pembuatan Movement Plan pada Robot Quadruped-legged," Proposal Tugas Akhir, ITS, 2024.

[6] W. Huang et al., "Language models as zero-shot planners: Extracting actionable knowledge for embodied agents," *ICML*, 2022.

[7] H. Ravichandar et al., "Recent advances in robot learning from demonstration," *Annual Review of Control, Robotics, and Autonomous Systems*, vol. 3, 2020.

[8] M. Ahn et al., "Do as I can, not as I say: Grounding language in robotic affordances," *arXiv:2204.01691*, 2022.

[9] D. Driess et al., "PaLM-E: An embodied multimodal language model," *ICML*, 2023.

[10] P. Lewis et al., "Retrieval-augmented generation for knowledge-intensive NLP tasks," *NeurIPS*, 2020.

[11] C. Rösmann et al., "Trajectory modification considering dynamic constraints of autonomous robots," *ROBOTIK*, 2012.

[12] P. Biber and W. Straßer, "The normal distributions transform: A new approach to laser scan matching," *IROS*, 2003.

[13] K. Koide et al., "Voxelized GICP for fast and accurate 3D point cloud registration," *ICRA*, 2021.

[14] DEEPRobotics, *Jueying Lite3 Pro User Manual*, V1.0.7-0, 2024.

[15] DEEPRobotics, *Jueying Lite3 LiDAR User Manual*, V1.0.7-0, 2025.

[16] M. Quigley et al., "ROS: an open-source robot operating system," *ICRA Workshop on Open Source Software*, 2009.